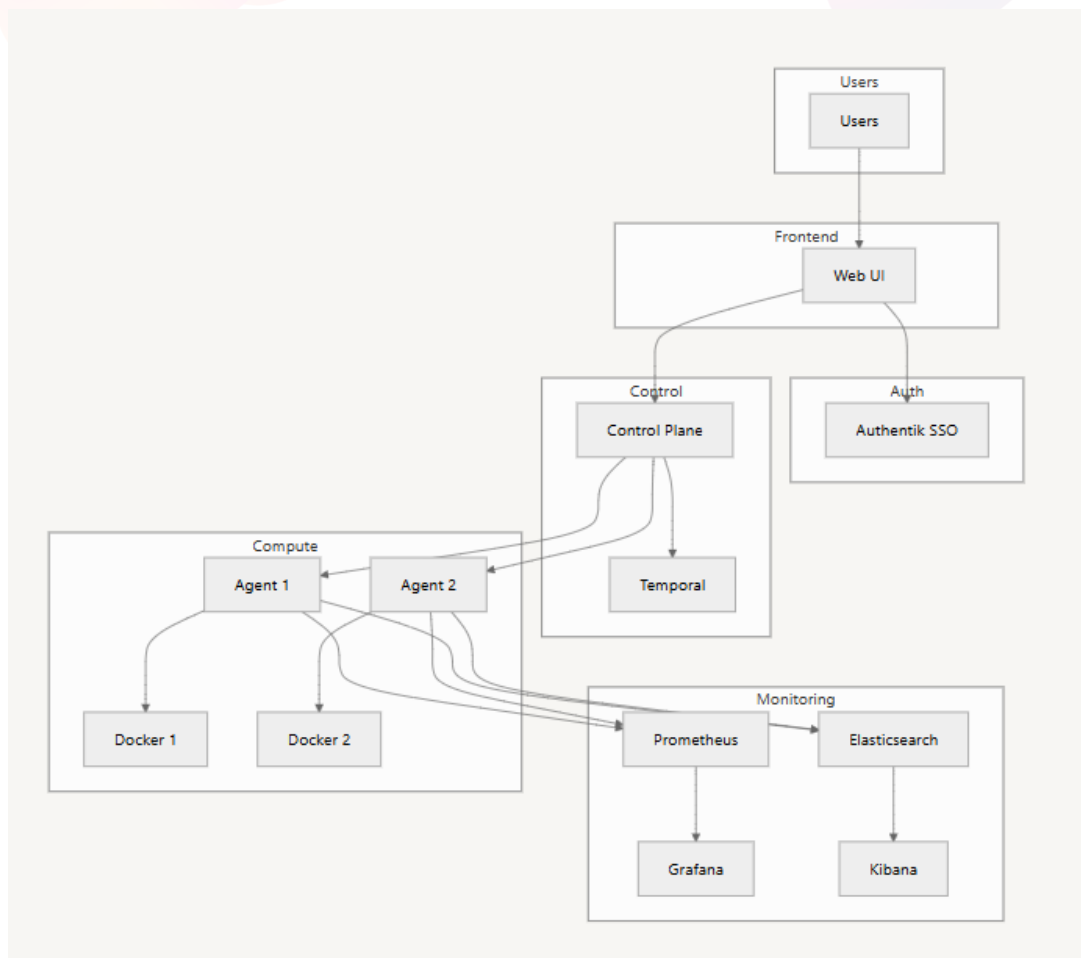


Final Exam: Build a Multi-Tenant Agent Cloud with SSO and Centralized Logging

Objective:

Design and implement a secure and scalable multi-tenant **agent-based compute platform**, where each tenant:

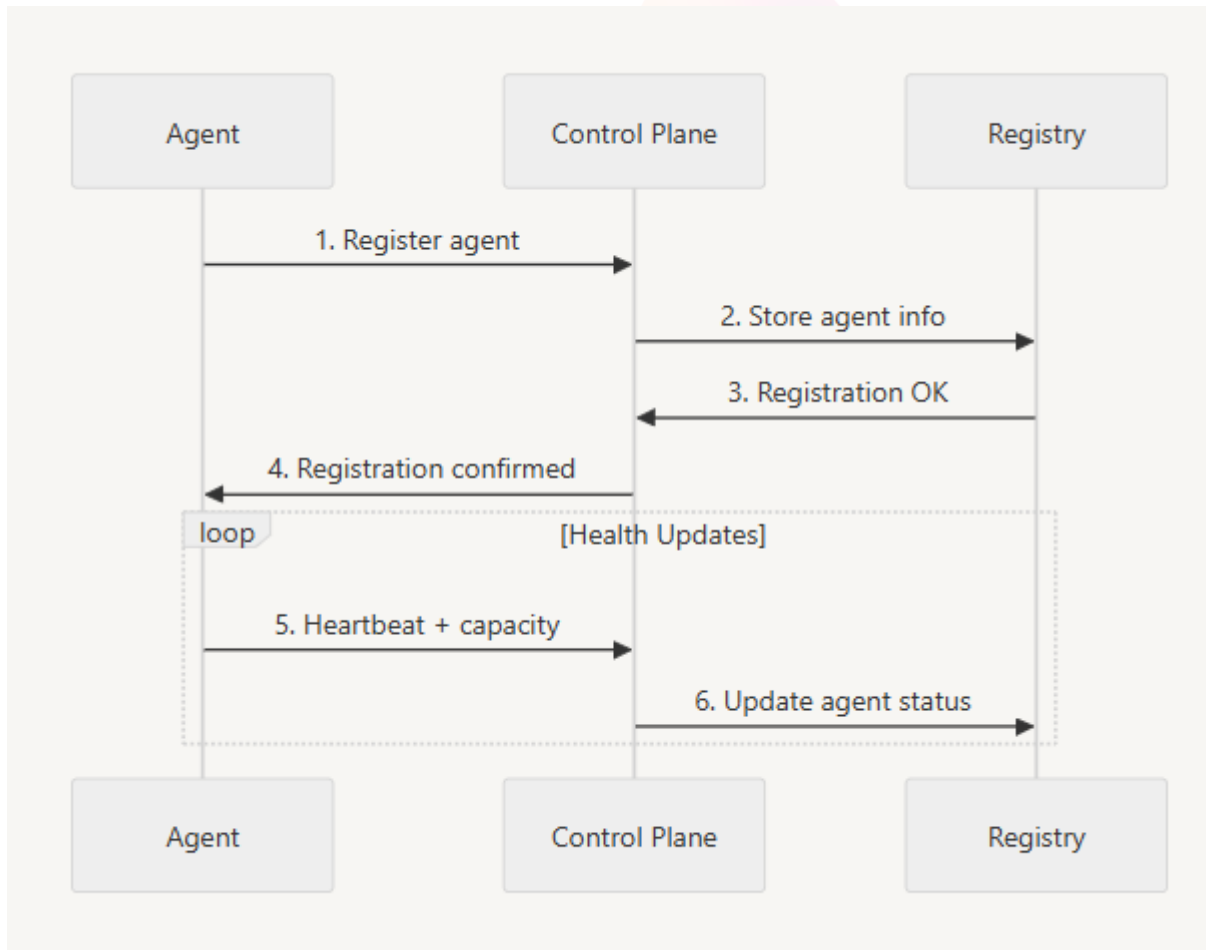
- Authenticates via **SSO using Authntik**
- Launches isolated containerized workloads via your **custom control plane**
- Has their logs collected in a centralized **Elasticsearch** setup
- Views resource metrics and live status via **Prometheus and a Web UI**
- Orchestrates container lifecycle using **Temporal workflows**
- Streams real-time logs using **Server-Sent Events (SSE)**



Tasks Breakdown

1. Multi-Tenant SSO with Authentik

- Deploy and configure Authentik as the identity provider
- Define at least 2 tenants (e.g., Team Alpha, Team Beta)

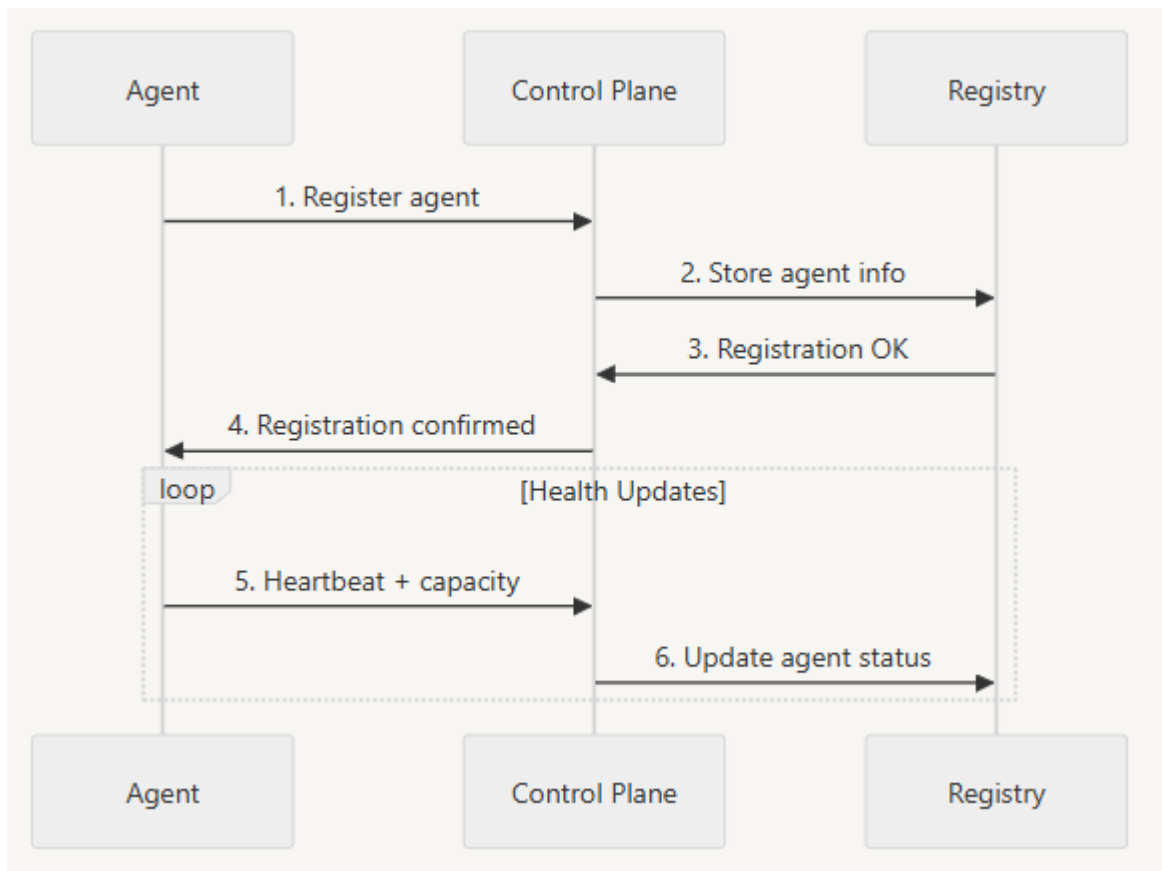


Authentication Flow

- Each tenant gets its own Authentik group
- Implement login flow to issue JWTs or access tokens to users
- Protect all APIs using Authentik-verified tokens

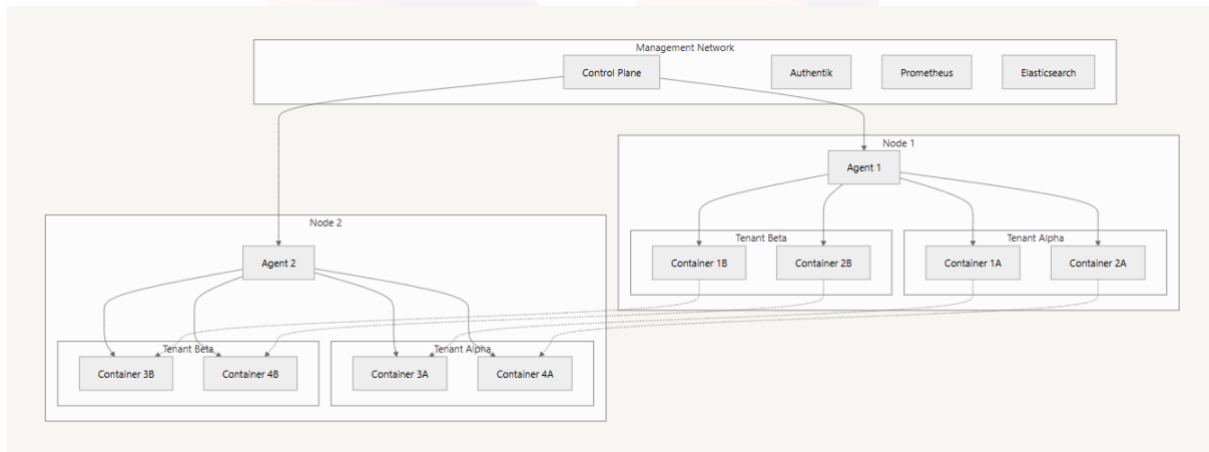
2. Agent Bootstrapping (Control Plane Integration)

- Build a lightweight agent (Golang or Python) per compute node
- On start, the agent must:
- Register with the central control plane
- Advertise available capacity and network info
- Operate on a **per-tenant VXLAN bridge** for isolation



3. Per-Tenant Container Launching

- Implement a secure API endpoint to launch containers
- Ensure launched containers:
 - Are connected to the correct tenant VXLAN
 - Run on isolated bridges per tenant
 - Are scheduled through your control plane logic

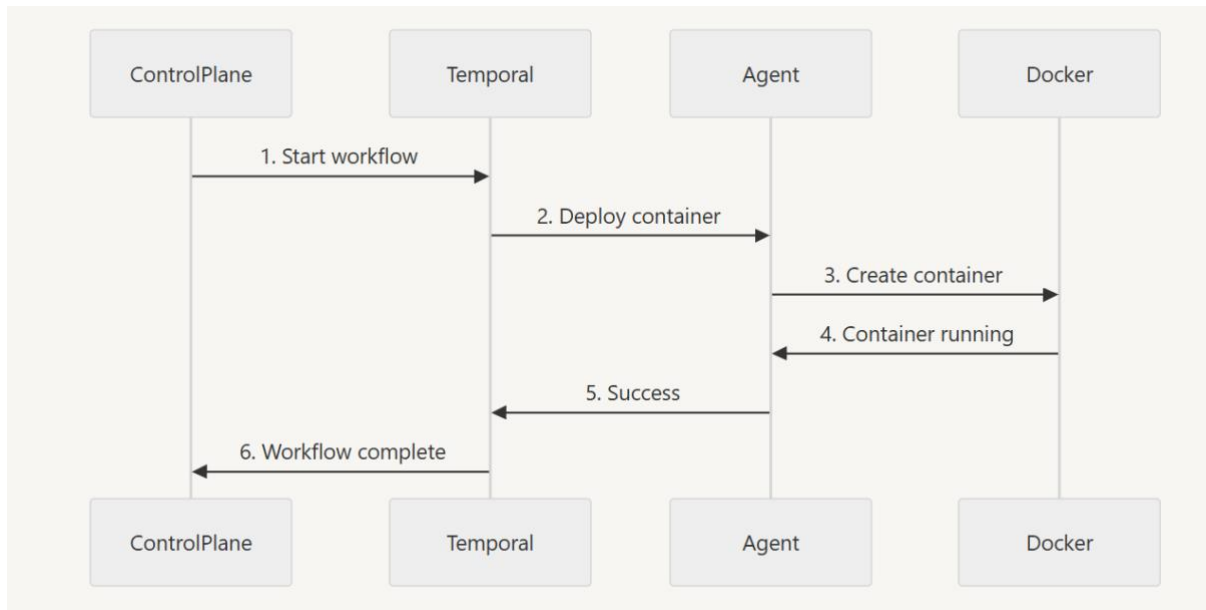


4. Centralized Logging with Elasticsearch

- Set up Fluent Bit or custom log shipper inside each agent
- All container logs must be:
 - Tagged with tenant_id, container_id, and node_id
 - Sent to a centralized Elasticsearch cluster
- Use Kibana to build per-tenant dashboards

5. Container Lifecycle Management with Temporal

- Integrate Temporal as the orchestration backend:



- Implement workflows for:
 - Launching container
 - Monitoring its health
 - Auto-termination on failure or TTL expiry

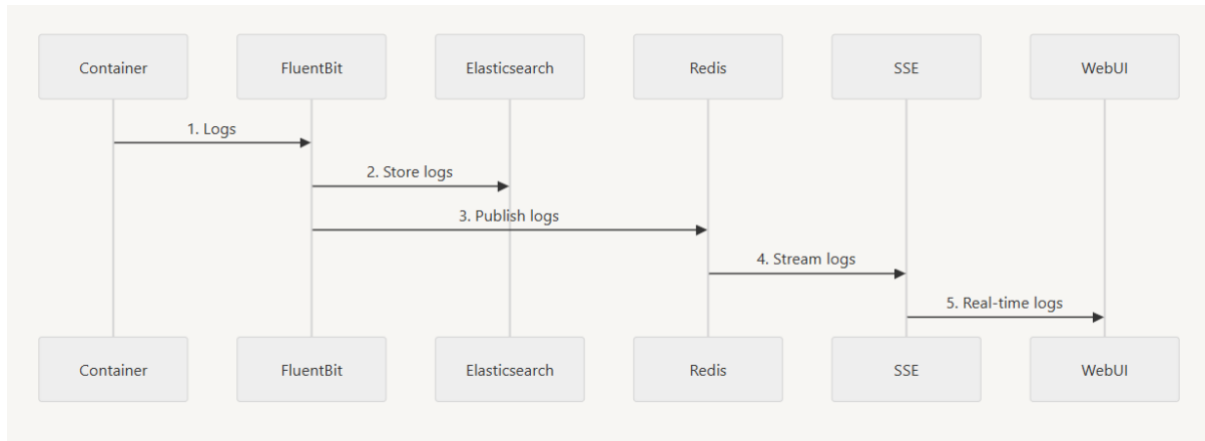
6. Metrics and Health Monitoring (Prometheus)

- Expose Prometheus metrics from each agent (e.g., CPU, memory, active containers)
- Set up node exporters or custom /metrics endpoint
- Aggregate and visualize metrics using Grafana

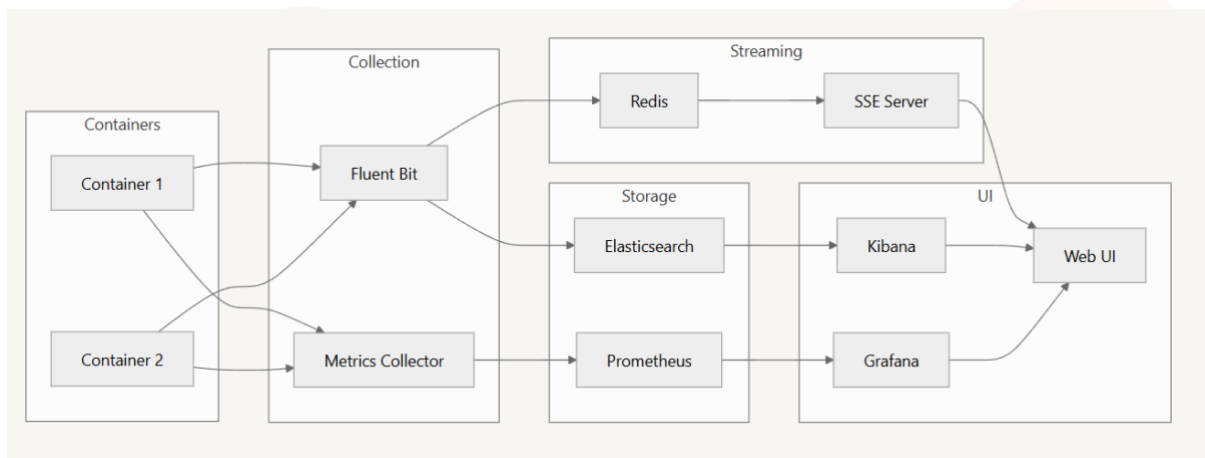
7. Web UI for Tenants

- Build a basic UI to show:
 - Logged-in user's active containers
 - Logs for each container (via Elasticsearch/Kibana proxy)
 - Live status and metrics
- Must show only the tenant's own data

8. Real-Time Log Streaming with SSE



- Build an SSE endpoint (`/logs/stream?tenant_id=xyz&container_id=abc`) in the backend
- Authenticate via Authentik JWT; authorize based on `tenant_id`
- Each agent publishes logs to Redis channels per container
- The SSE server subscribes to these channels and streams log lines to clients
- Implement a frontend listener using JavaScript `EventSource`



Evaluation Criteria

- Correctness of **Auth** and **multi-tenancy isolation**
- Reliability of **agent registration** and **container deployment**
- Accuracy and structure of **log data in Elasticsearch**
- Working **Temporal integration** with clear workflows
- Functional **Prometheus metrics endpoint** and **Grafana dashboard**
- Clean, secure, and tenant-isolated **web interface**
- Functioning **SSE log streaming** tied to tenant/container.

=====